

YouTube Shorts Recommendation System

Complete End-to-End Project Guide

Resume Project Work Experience Description

Machine Learning Engineer - Shorts Recommendation System | YouTube (Google) | Feb 2024 - Present

- Built a real-time recommendation pipeline for YouTube Shorts, leveraging deep learning models and session-based ranking to increase user engagement by 21%.
- Developed a multi-objective ranking model balancing watch time, likes, and creator diversity using Transformer-based encoders and candidate scoring pipelines.
- Integrated implicit feedback signals (rewatch rate, skips, scroll velocity) to enhance personalization using RNNs and GNNs.
- Designed a candidate generation system combining collaborative filtering, trending models, and graph-based similarity scoring.
- Deployed pipeline using TensorFlow Extended (TFX) and Google Cloud Dataflow, optimized for sub-100ms latency on mobile clients.
- Implemented automated A/B testing and performance monitoring with Kubeflow Pipelines and Vertex AI, reducing model update cycle by 40%.

System Architecture & Tech Stack Overview

- The system follows a modular design with multi-stage ranking pipeline:
 - Candidate Generation (ANN Search via FAISS/ScaNN)
 - Ranking Models (Transformer / MLP) trained on watch sessions
 - Real-time personalization using Redis or BigQuery features
 - Model Serving via TensorFlow Serving or Vertex AI Endpoint
 - Monitoring via W&B, Prometheus, and retraining through GitHub Actions / Vertex Pipelines

Project Roadmap: Week-by-Week Execution Plan

Week 1: Business Understanding & Problem Definition

Tasks:

- Research YouTube Shorts recommendation objectives (maximize user session length, engagement).
- Study previous RecSys challenges (e.g., cold start, multi-objective ranking).
- Identify key KPIs: Watch Time, Retention Rate, Daily Active Users (DAU).

Deliverables:

- Clear problem statement: personalized Shorts recommendation at scale.

- Define success metrics for offline training and online serving.

Reference:

- Deep Neural Networks for YouTube Recommendations (Covington et al.)

Checklist:

- Clear articulation of business problem
- Metrics baseline assumptions documented

Week 2: Data Schema Design & Ingestion Pipeline Setup

Tasks:

- Define schema: user features, video features, interaction features.
- Simulate synthetic user-video interaction logs (views, likes, shares).
- Build data ingestion script (Python + Pandas) to preprocess raw logs.

Deliverables:

- Cleaned and normalized CSV datasets (train/test splits).
- Documented feature dictionary.

Reference:

- Feature Engineering Best Practices - Google MLOps

Checklist:

- Complete user/item feature tables
- Missing value handling strategy documented

Week 3: Candidate Generation Model (First Stage Retrieval)

Tasks:

- Implement simple collaborative filtering baseline (e.g., implicit feedback matrix factorization).
- Train ANN (Approximate Nearest Neighbor) index using FAISS.
- Evaluate Recall@K for candidate retrieval.

Deliverables:

- Saved user/item embeddings.
- Working FAISS candidate retrieval pipeline.

Reference:

- FAISS Official Documentation

Checklist:

- >80% Recall@1000 in offline validation
- Embeddings quality visualized (t-SNE or PCA)

Week 4: Ranking Model - Baseline Training

Tasks:

- Design ranking model: MLP (multi-layer perceptron) with user/item features.
- Train with standard loss (e.g., cross-entropy or pairwise loss).
- Validate ranking effectiveness with NDCG, MRR metrics.

Deliverables:

- Trained baseline ranking model.
- Offline evaluation report.

Reference:

- Deep Learning for Recommender Systems (YouTube)

Checklist:

- Top-10 NDCG > 0.35 in offline test set
- Loss convergence curve plotted

Week 5: Feature Engineering Improvements

Tasks:

- Add additional features: temporal recency, user session embeddings, video popularity decay.
- Engineer cross features (e.g., user demographics × video category).
- Normalize numerical features, encode categorical ones.

Deliverables:

- Updated feature pipeline.
- Re-trained improved ranking model.

Reference:

- TFX Transform Documentation

Checklist:

- Feature importance chart created
- New features lead to at least +5% AUC lift

Week 6: Cold Start User & Item Handling

Tasks:

- Implement fallback strategies for new users (e.g., trending videos, demographic cohort modeling).

- Use side information (video metadata) for new video embeddings.
- Train separate small ranking model for cold start scenarios.

Deliverables:

- Cold start handling module integrated.
- Separate evaluation on cold-start samples.

Reference:

- Solving Cold Start Problem in RecSys - ResearchGate Overview

Checklist:

- Offline cold-start NDCG evaluated separately
- $\geq$ baseline performance on unseen users/videos

Week 7: Serving Pipeline Setup (Prototype)

Tasks:

- Build serving pipeline with FastAPI or lightweight Flask server.
- Load trained candidate generator + ranking models.
- Serve Top-N recommendations via API call (input: user_id).

Deliverables:

- Working end-to-end local server for recommendations.
- Example JSON input/output documented.

Reference:

- FastAPI Tutorial

Checklist:

- Latency < 300 ms per API call
- Serving logs enabled

Week 8: Evaluation System & A/B Test Simulation

Tasks:

- Build A/B Test Simulator: assign users randomly into baseline vs experimental models.
- Track watch time, click rate, session length per group.
- Perform statistical significance tests (t-test, bootstrap).

Deliverables:

- Simulated A/B Test Results Report.
- Conclude whether the new model significantly outperforms the baseline.

Reference:

- Practical Guide to A/B Testing in ML

Checklist:

- p-value analysis conducted
- Minimum Detectable Effect (MDE) calculated

Week 9: Monitoring & Debugging Pipeline

Tasks:

- Implement batch logs: served candidate lists, clicks, feedback.
- Build dashboards (e.g., Grafana, Streamlit) to monitor CTR, engagement trends.
- Set alert thresholds for model performance drops.

Deliverables:

- Prototype monitoring dashboard - Daily logs generated and parsed

Reference:

- ML Monitoring Best Practices (Google)

Checklist:

- Dashboard shows CTR over time
- Alert for CTR drop >10%

Week 10: Final Packaging & Documentation

Tasks:

- Consolidate codebase into clean modules (data_loader.py, model.py, server.py).
- Write README.md for project setup & usage instructions.
- Prepare presentation slides summarizing project architecture, results, and learnings.

Deliverables:

- Complete project repo (ready to publish on GitHub)
- Final project presentation deck

Reference:

- GitHub Project Structure Best Practices

Checklist:

- README includes Setup / Run / Evaluate sections
- Codebase ready for recruiter/manager to review

Key ML Design Decisions & Interview Talking Points

Design Trade-offs:

- Multi-objective loss design: balancing watch time vs. creator diversity
- Cold start handling: hybrid scoring using content + metadata
- Latency vs. performance: simplified model for real-time top-N reranking

Interview Talking Points:

- Mention candidate generation as scalable ANN-based retrieval using FAISS
- Highlight ranking model trade-offs and offline metrics: NDCG@10, Recall@100
- Explain real-time architecture and use of session features for reranking
- Talk about A/B test setup and CI/CD with GitHub Actions and Vertex AI
- Tie project metrics to product KPIs: 20% watch time lift, <100ms latency

Metrics & Benchmarking Strategy

Offline Metrics:

- Precision@K, Recall@K, NDCG@K
- Diversity Score (creator entropy)
- Average Watch Time, Skip Rate

Online Metrics (A/B Testing):

- Engagement Lift (% increase in watch time per user) - Click-Through Rate (CTR) on Shorts preview cards
- Session Duration & Repeat Visits

Latency Benchmarks:

- Candidate generation: <300ms
- Final ranking inference: <100ms
- End-to-end delivery: <500ms

Interpretability:

- SHAP values to explain model behavior
- Actionable diagnostics for product teams

Common Mistakes & Debugging Tips

- Model overfits short videos with extreme watch rates. Fix: Clip engagement metrics and balance data.
- Candidate generation returns redundant or stale content. Fix: Filter with content freshness + history.

- UI doesn't reflect updated recommendations after feedback. Fix: Use session-aware reranking/cache busting.
- Offline metrics look good, but live metrics drop. Fix: Validate with holdout/live A/B test.
- Feedback toggles (e.g., replays, skips) are noisy. Fix: Smooth signal using EMA or session features.

Cold Start Strategy Section

- Use user demographic features to cluster new users.
- Boost popular trending videos to new users.
- Apply self-supervised pre-training on video metadata for unseen content.
- Fine-tune a small model specifically on few-shot cold start data.

Suggested Study Materials

Recommendation System Theory

- Google Developers - Recommendation Systems Overview:

<https://www.youtube.com/watch?v=ZspR5PZemcs>

- Coursera: Recommender Systems Specialization:

<https://www.coursera.org/specializations/recommender-systems>

Serving & Pipelines

- TFX: TensorFlow Extended for ML Pipelines:

<https://www.tensorflow.org/tfx>

- DeepLearning.AI: Deployment of ML Models:

<https://www.deeplearning.ai/programs/machine-learning-engineering-for-production-mlops/>

Infrastructure & Metrics

- Kubeflow Pipelines Documentation:

<https://www.kubeflow.org/docs/components/pipelines/>

- YouTube Recommendations at Scale (Google Research):

<https://research.google/pubs/pub44806/>